

gnss-scope 実装設計（202607）

実装の設計書（構成・データモデル・実装上の注意）。役割分担は次のとおり。

文書	内容
funcspec-202607.md	機能仕様（何をするか・仕様値）
usersGuide-202607.md	利用者の手引（操作手順・画面の読み方）
design-202607.md （本書）	実装設計（画面実装・データモデル・モジュール構成・リファクタ記録）
algospec-202607.md	算出式・自動判定ロジックの詳細
hardware-202607.md	機器・ファーム（Pico W / MAX-M10S / BLE）の仕様

本書は、前身 [gnss-tracker](#)（地理院地図の全画面表示がメイン）を [gnss-scope](#) として再構築した際の設計を出発点としている。受信パイプライン（BLE → 行復元 → NMEA パース → エポック確定 → 受信品質統計）の考え方は前身から引き継いでおり、**変更点は UI 構成・記録モデル・解析の対象**である。

1. 方針

- **地図をメインにしない。** 地図は5タブのうちの1つ。役割も「ライブ地図」から「**load した記録の GNSS 値をマーカー表示する**」へ移す（ライブ現在地表示は副機能として残す）。
- **記録は静止点のばらつき評価に一本化。** 1回の記録 = 1地点。旧「スナップショット記録（1点だけ即保存）」は廃止し、[record](#) → [stop](#) → [save](#) に統一。旧「静的測位記録」の収集・集計・収束自動停止ロジックはそのまま流用する。
- **解析は SkyPlot / SNR / DOP / DRMS の4本立て**とし、ライブと読込データの双方を同じ画面で見る。

2. 画面構成

ステータスバー（常設）	接続状態ドット / 状態 / REC / fix / 最終受信
ページ（タブで排他・全画面）	接続 / 記録 / 解析 / 地図 / 設定
タブバー	

旧構成（地図全画面 + 下部ボタンで開くボトムシート）を廃止し、[.page](#) の排他表示に変更した（[initTabUI.js/view-utils.js](#)）。非表示中の `<canvas>` と Leaflet はサイズが 0 になるため、**タブ表示時に描き直す**：

タブ	表示時の処理
記録	<code>recordUI.onShow()</code> （散布図の再描画）
解析	<code>analysisUI.refresh()</code> （現在のソースで全再描画）

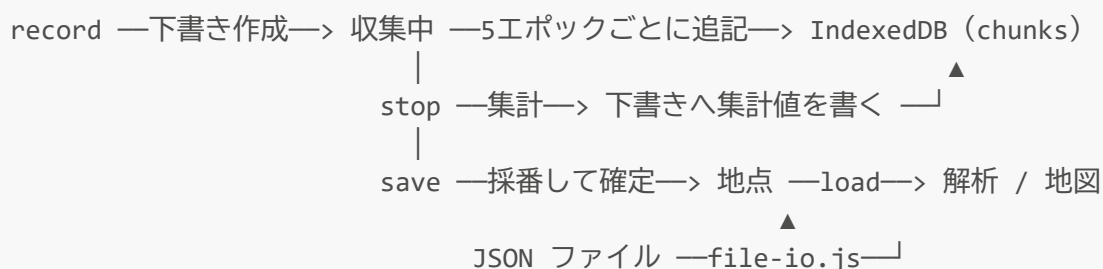
タブ 表示時の処理

地図 `mapUI.onShow()` (`invalidateSize()` + 読込データへフィット)

設定 `settingsUI.refreshVersion()` + 端末内データ量の数え直し

`CanvasView.redraw()` を追加し、「サイズ 0 のまま描いた分の描き直し」を共通化した。

3. 記録モデル (record → stop → save → load)



- **record** は **DB に下書きを作ってから** 収集を始める。作れなければ記録しない（残せない記録を始めても仕方がない）。先に親を作るので孤児チャンクが発生しない。
- 収集中は **5 エポック（≒5 秒）ごとに chunks へ追記**する。保存を押す前に落ちても直近の追記まで**は残る**のがこの設計の要点。詳細は 4. の「実データを分割して追記する」。
- **stop** は集計して pending を返し、記録タブに散布図＋凡例＋測位結果を表示する。DB への書き込み（残りのバッファ＋集計値）は**描画の後に**走らせる（`await` しない）。停止理由（`converged / timeout / maxEpochs / manual / stalled / storageError`）と、収束判定を働かせていたか（`autoStop`）を `summary` に持つ。後者は停止サマリで「未収束」と「収束判定なし」を書き分けるために要る（`algospec-202607.md` 3-5）。
- **save** は**確定**操作。地点番号（`pointNo`）を採番し、地点名（既定 `yyyy-mm-dd-xx`）とメモを付けて `status` を `confirmed` にする。**実データには触らない**（メタの更新だけ）。確定した記録をそのまま「読込データ」に設定する（解析タブ・地図タブへ即反映）。
- 収束自動停止（`algospec-202607.md` 3.）は維持。自動停止も `stop` と同じ経路を通る。
- 記録中は Wake Lock を取得する。

下書きを挟む理由

停止した時点で「地点」にしてしまうと、測り直した失敗記録が地点番号を消費して番号が飛び、対応表 CSV・バンドルJSON・調査日の平均DRMS にも混ざる。かといって停止まで DB に書かないと、保存を押す前に落ちたときに全損する。**書くのは早く、地点にするのは遅く**が下書きの役割。

- 下書きは `pointNo: null`。`nextPointNo` は `Number.isFinite(s.pointNo)` を見るので、下書きは採番から自動的に外れる。
- 集計・エクスポートは `isConfirmed(session)` (`status !== 'draft'`) で絞る。
- 一覧では調査日の先頭に出る（`pointNo ?? 0` の並びで先頭に来る）。再起動後もそこから確定できる。
- 中身のある下書きは自動削除しない。ただし起動時に `chunks` を 1 件も持たない下書き（最初の追記より前に落ちたもの）だけは片付ける（`cleanupEmptyDrafts`）。失うものが無いため。

打ち切りは「画面が隠れたか」ではなく「データが来ているか」

記録中だけ 1 秒間隔のタイマーを回し、最後のエポックから **10 秒** (`recorder.js` の `STALL_MS`) 過ぎたら `stop('stalled')` で打ち切る。

- 停止判定 (収束・`maxSec`・`maxEpochs`) は `addEpoch` の中にしか無い。エポックが届かなくなると判定自体が走らず、記録が凍る。旧版の `visibilitychange` による中断停止はこのうち画面 OFF の場合しか塞いでいなかった (BLE 切断・電池切れ・fix 喪失では凍ったまま)。
- 判定の基準は「蓄積対象になったエポックを受けた時刻」。`addEpoch` は fix の無いエポックを弾くので、**fix 喪失も途絶として扱う** (測位できていない状態で記録を続ける意味は無い)。
- 記録に使う端末は**記録専用**とする前提。地図・写真は別の端末で見る。これにより「短い離脱で毎回やり直しになるのを避ける」ための猶予 (旧 5 秒) が不要になった。
- 10 秒にしているのは、BLE の瞬断が自動再接続 (`transport.js`) で復帰するまでの猶予。
- `Recorder` / `DeviceGnss` に `paused` 状態は持たない。収集中かどうかは `current` の有無だけ。
- 秒数と点数を併記する停止サマリ (**180秒/50点** など) が、欠けたぶんの手掛かりになる。

サンプルに衛星リストを含める

保存サンプル (`toSample`) に `satellites: [{sys, prn, elev, azimuth, snr, used}]` を追加した。これにより **保存後・取込後でも任意エポックの SkyPlot / SNR を再現**できる (解析タブのスライダー)。容量は 1 エポックあたり衛星30個程度 = 数KB、180 エポックで数百KB程度に収まる。

4. データモデル (IndexedDB: `gnssScopeDB v3`)

DB 名は旧 `gnssDB` から変更した (旧アプリのデータと混ざらないようにするため)。

1日に何十地点もまわって 2系統 (GNSS受信機の NMEA と Android内蔵の測位) を比べる使い方に合わせ、**調査日 → 地点 → 実データ**のツリーにしている。

```

surveys    '2026-07-31'                ← 根: 1日の調査
├── sessions rec_<開始ms> + surveyId / pointNo / status ← 枝: 1地点 (20地点なら20件)
│   ├── points  sessionId + surveyId / pointNo / stats ← 葉: その地点の集計値
│   └── chunks  sessionId + seq                        ← 葉: 実データ (5エポックずつ)
│       ├── samples[]  GNSS受信機のパース済みエポック
│       ├── rawNmea[]   GNSS受信機の生NMEA行
│       └── deviceSamples[] Android内蔵GNSS の測位

```

ストア	キー	内容
<code>surveys</code>	<code>id</code> (yyyy-mm-dd)	{ <code>id</code> , <code>createdAt</code> , <code>updatedAt</code> , (<code>exportedAt</code>) }
<code>sessions</code>	<code>id</code> (bySurvey index)	{ <code>id</code> , <code>type</code> : 'record', <code>status</code> : 'draft' 'confirmed', <code>surveyId</code> , <code>pointNo</code> , <code>label</code> , <code>memo</code> , <code>createdAt</code> , <code>endedAt</code> , <code>window</code> , <code>summary</code> , (<code>confirmedAt</code> , <code>importedAt</code> , <code>sourceId</code> , <code>sourcePointNo</code>) }
<code>points</code>	<code>id</code> (bySession index)	{ <code>id</code> , <code>sessionId</code> , <code>surveyId</code> , <code>pointNo</code> , <code>kind</code> : 'record', <code>stats</code> , (<code>deviceStats</code>) }

ストア	キー	内容
chunks	id (bySession index)	{ id:_, sessionId, seq, samples[], (rawNmea[], deviceSamples[]) }
photos	id (bySession index)	{ id:_ph, sessionId, seq, blob, bytes, w, h, addedAt }
settings	key	tileCacheMeta のみ（事前DLしたタイルの版・種別・日時）

- `status: 'draft'` = 停止したがまだ地点になっていない / `'confirmed'` = 保存（確定）済み。判定は `isConfirmed(session)` (`storage.js`) に集約する。下書きは `pointNo: null`。
- `summary: { lat, lon, altMSL, count, drms, cep50, cep95, stopReason, autoStop, rxStats, rawLines, rawTruncated, (deviceDrms, deviceCount) }`
 - 記録中は追記のたびに `count / rawLines / deviceCount / bytes` だけを更新する。停止せずに落ちた記録でも「どこまで録れているか」が一覧に出るようにするため。
 - `bytes / photoBytes / photoCount` は端末内サイズの集計用（下記）。
- 調査日は**テキストを持たない**（日付そのものが識別子で、書くことは地点側にある）。`exportedAt` はバンドルJSON を書き出した日時で、`updatedAt` を下回れば「未書き出し」に戻る。
 - `autoStop` は収束判定を働かせていたか。停止サマリの但し書きの出し分けに使う。この項目を持たない旧データは「未収束」に寄せて表示する。
- `stats: computeStaticStats()` の戻り値（中心・中央値・東西/南北標準偏差・DRMS/2DRMS・CEP50/CEP95・標高平均/標準偏差・fix内訳・平均DOP・平均衛星数・平均C/N0・散布図オフセット）
- 設定キーは旧 `staticMinSec` などから `minSec` 等へ改名（記録が静的測位専用ではなくなったため）。
- 設定タブの値は **settings ストアへ保存しない**（既定値は `js/constants.js`）。端末に残った古い値が新しい既定値を覆い隠すのを避けるため。`settings` ストアに残るのは、Cache Storage の実体と対応する端末側の状態（`tileCacheMeta`）だけ。旧版が書き込んだ設定キーは読まれず放置される。

実データを分割して追記する（chunks）

停止時に `rawNmea` を 1 回で書く設計だと、最大 1.6MB の structured clone がメインスレッドに 乗るうえ、**そこへ至る前に落ちれば全損する**。そこで実データを `points` から外し、記録しながら `chunks` へ 5 エポックずつ追記する。

- **粒度はエポック境界**。生行は `addRawLine`、パース結果は `assembler.add` と別経路で流れ（`app.js`）、`addEpoch` の時点が「このエポックと、その間に受けた生行」が揃う唯一の タイミング。ここで区切れば 2 系統の対応がずれず、1 トランザクションにまとめられる。
- 1 エポックごと（1 tx/秒）でも動くが、トランザクションが 5 倍になる割に、得られるのは「失う量が 5 秒 → 1 秒」だけ。60 秒の記録の末尾 5 秒を失っても集計はほとんど動かない。
- **追記のみ。既存チャンクは読まない・書き直さない**。これで総シリアルライズ量が $O(\text{全バイト数})$ に収まる（成長中の配列を毎回書き直すと $O(n^2)$ になる）。
- 1 チャンクは約 20KB。structured clone はサブミリ秒で、毎エポック既に走っている `computeStaticStats` と散布図の canvas 再描画のほうがずっと重い。受信経路を待たせないよう **await しないで呼ぶ**。
- フラッシュ後は生行バッファを捨てられるので、メモリに残るのは現在のバッファだけ。`MAX_RAW_LINES` はメモリ圧迫の保険ではなく **DB 容量の保険** という意味に変わった。

- 失敗したぶんはバッファへ戻して次のチャンクで再試行し、連続 3 回で `stop('storageError')`。
- **fix 喪失中はフラッシュが走らない** (`addEpoch` が弾くため) が、途絶停止が 10 秒で打ち切るので溜まる量には上限が付く。時間ベースのフラッシュは併用しない。
- 読み出しは `getPointsBySession()` がチャンクを `seq` 順に結合し、**従来と同じ形の point** (`samples / rawNmea / deviceSamples` 入り) を返す。解析タブ・地図・エクスポートの呼び出し側は分割を意識しない。取込データも 1 チャンクとして入れ、読み出し経路を 1 本に保つ。

端末内のデータ量を自分で数える

`navigator.storage.estimate()` はオリジン全体を返すため、事前ダウンロードした地図タイル (Cache Storage に数十MB) が混ざって**記録データの量が分からない**。そこで自前で数える。

- 数えるのは**フラッシュのとき** (`JSON.stringify(data).length` を積む)。停止時に数えると書き終えたチャンクを読み戻すことになる。20KB の JSON 化は誤差の範囲で済む。
- 写真は `blob.size` をそのまま足す (こちらは正確)。
- 合計は `sessions` だけで数えられるよう、`summary.bytes / photoBytes` に持たせる (`storage.getStorageUsage()`)。地点を消せば自動で辻褄が合う。
- **JSON の文字数は IndexedDB の実使用量ではない** (structured clone の形式とインデックスのオーバーヘッドが乗る)。同じオーダーには収まるので警告の判断には足りるが、表示は「約」を付ける。
- しきい値は `constants.js` の `STORAGE_LIMITS` (警告 50MB / 想定上限 100MB)。超えても**記録は止めない** (現場で測れなくなる方が困る)。警告を出し続けて整理を促す。

地点の写真

`photos` ストアへ**縮小済みの JPEG を Blob のまま入れる** (base64 は 33% 増えるうえ、表示のたびにデコードが要る)。`chunks` と同じ `bySession` の形にしてあるので、削除・容量集計・読み出しの経路をそのまま流用できる。

- 縮小は**取り込む時点**で行う (`photo-ui.js`)。後から縮められないため。長辺を設定値に収めたうえで、200KB に入るまで JPEG 品質を段階的に落とす。
- EXIF の向きは `createImageBitmap` の `imageOrientation: 'from-image'` に任せる (自前で EXIF を読むと端末ごとの差分を抱え込む)。
- 枚数とバイト数は `session.summary` にも複写する。一覧の各行や容量集計で `photos` ストアを引き直さずに済ませるため。
- 表示は `URL.createObjectURL`。貼り替えのたびに **`revokeObjectURL` する** (貯めると端末を圧迫する)。

地点の対応をどう保証するか

2系統を同じチャンクに入れるのが第一の担保で、これで「どちらの系統のどのデータが 対か」は構造上ずれない。その上で、後から見て「本当に同じ時間に測ったか」を確認できるよう `session.window` (`js/survey.js` の `buildWindow`) を残す。

フィールド	内容
<code>startedAt / endedAt / durationSec</code>	record → stop の記録区間 (端末時計)

フィールド	内容
<code>gnss</code>	受信機のエポックが実際に取れていた範囲 <code>{startedAt, endedAt, durationSec, count}</code>
<code>gnssUtc</code>	同じ範囲を GPS時刻(UTC)で見たもの
<code>device</code>	Android内蔵の測位が取れていた範囲
<code>overlap</code>	<code>{ overlapSec, coverGnss, coverDevice, deviceInRecording, deviceTotal }</code>
<code>clockOffsetMs</code>	端末時計 - GPS時刻。2系統を同じ時間軸へ並べ直すときの補正量
<code>deviceLagMs</code>	Android内蔵の 受信時刻 - 測位確定時刻。古い fix を返していないかの目安

時刻軸の扱いに注意が要る。受信機のサンプルは `t=GPS時刻 / recvAt=端末時計`、Android内蔵は `t=OS` が測位を確定した時刻 / `recvAt=アプリが受け取った時刻`で、**`t` どうしは別の時計**（内蔵は `maximumAge:0` でも数秒前の fix を返すことがある）。区間の突き合わせは両者が共通に持つ `recvAt` で行い、`t` とのズレは `clockOffsetMs / deviceLagMs` として別に残す。

`overlap.deviceInRecording` は**記録区間**を基準に数える。受信機の最初のエポックは 複数センテンスが揃うまで確定せず1秒強かかるため、`gnss` の範囲で数えると 記録開始直後に届いた内蔵のサンプルを取りこぼす。また、静止中は OS が更新を間引いて 内蔵が1～数点しか来ないことがあり、割合（`coverDevice`）だけでは対応を判断できない。

地点番号 `pointNo` は同じ `surveyId` の中の連番で、**欠番は再利用しない**（最大値+1）。一度出した対応表と番号がずれないようにするため。

生NMEA行の保存

設定 `saveRawNmea`（既定 ON）のとき、受信した行を `chunks[].rawNmea = [{ t, line }]` にそのまま残す。パースを通す前に分岐させるので、チェックサムNG行や `$PPICO` も含まれる（「受信したそのまま」を残す意味）。後から RTKLIB などの別ツールで再解析できる。

容量は 1Hz・10～20行/秒で 1地点（30～180秒）あたり 40～250KB、20地点/日で 3～5MB 程度。上限 20000 行（`MAX_RAW_LINES`）を超えたぶんは捨て、本数を `summary.rawTruncated` に残して 記録が途中までであることを隠さない。

v3 / v4 での作り直し

バージョンを上げるたびに既存ストアを全部 `deleteObjectStore` して作り直す。開発中のデータしか 無いため、**移行コードを持たないほうが結果的に安全で簡単**という判断。これにより v1 → v2 の移行処理（`migrateToV2`）と、旧形式（`point` に配列を直接持つ）のフォールバック 分岐を持たずに済む。全レコードが `status` を持つ前提にもできる。

旧版を開いたままのタブがあるとアップグレードはブロックされる。**既にデプロイ済みの v2 のコードには `onversionchange` が無いため、これは避けられない。** `onblocked` で「他のタブを閉じて再読込」を案内する。今後のアップグレードのために `db.onversionchange` で自分の接続を閉じるようにした。

Android内蔵GNSS の並行取得（比較データ）

`deviceSamples / deviceStats / summary.deviceDrms` は **並行取得した記録にだけ足す**（取得していない記録の形は従来のまま＝後方互換。取込側も任意扱いで読む）。

- `deviceStats` は `computeDeviceStats()` の戻り値 = `computeStaticStats()` の全項目 + `avgAccuracy / dupCount / offsetFromRef {e, n, distM, bearingDeg}`。ばらつきの定義を受信機側と共有するため、集計本体は `computeStaticStats` をそのまま使う。
- `offsetFromRef` の基準は **GNSS受信機の中心**。散布図で共通原点に重なる際の平行移動量 そのものなので、描画側は差分を再計算せず `offsetFromRef.e / .n` を足すだけでよい。
- `dupCount`（直前と完全同一座標の点数）は保存後・取込後も再計算できるよう サンプル列から算出する（フラグとしてサンプルに持たせない）。

エクスポート / インポート

地点ごと（`file-io.js`）：

- CSV: 1行1エポック。列に `snr_avg_used_dbhz`（使用衛星の平均 C/N0）と `recv_at_local` を持つ。末尾に地点の識別（`survey_id / point_no`）と集計値をコメント行で付ける。Android内蔵は行として混ぜず、`# device_*` のコメント行で比較値のみ出す。
- GPX: 記録の中心を `wpt`、生エポック群を `trk` として出力（GNSS受信機のみ）。
- NMEA: `point.rawNmea`（チャンクを結合したもの）の行をそのまま `.nmea` として出力（先頭に地点を示すコメント行）。
- JSON: `{ app:'gnss-scope', format:1, exportedAt, session, point }`。

調査日ごと（1日20地点をまとめて持ち出す）：

- 対応表CSV: **1行1地点**で2系統を横並びにした表（`compareRow / COMPARE_HEADER`）。列名の接頭辞 `gnss_*` / `dev_*` が系統を表す（機種名は入れない）。先頭2列が `survey_id / point_no` で、これが地点の対応そのもの。ばらつき（DRMS / CEP / 標準偏差）に加え、`center_offset_m · drms_ratio_dev_over_gnss`、対応の検証列（`overlap_s / cover_*_pct / dev_in_record_window / clock_offset_ms / dev_fix_lag_ms`）を持つ。2系統比較の入口はこのファイル。
- バンドルJSON: `{ app:'gnss-scope', format:2, exportedAt, survey, points:[{session, point}] }`。`point` を丸ごと入れるので生NMEA・生エポック・Android内蔵の測位が地点ごとに揃ったまま出る。

取込（`importSessionFile`）は単体（format 1）・バンドル（format 2）を同じ入口で受け、常に `{session, point}` の配列を返す。取込時は **必ず新 id を採番**し（同じファイルの二重取込でも上書きしない）、元 id は `sourceId`、元の地点番号は `sourcePointNo` に残す。調査日は元データの日付をそのまま使い（同じ日の測定は同じ調査日に集まる方が突き合わせやすい）、地点番号だけ取込先で採り直す。`stats` が欠けている JSON は `computeStaticStats` で再計算する。

5. 解析タブ

表示	ライブ	読込データ
SkyPlot / SNR	最新エポック	スライダで選んだエポック（保存済み衛星リストから再現）
DOP（バー＋数値・測位モード・衛星数・精度ソース・GST σ）	最新エポック	同上

表示	ライブ	読込データ
DRMS（散布図＋集計テキスト）	記録中／直近の記録の暫定集計	読込データの確定集計
DRMS の比較系列（Android内蔵）	同区間の暫定集計（記録中から見える）	<code>point.deviceStats</code>

- レイアウトは2列グリッド（`.an-grid`）。上段 SkyPlot / DRMS、下段 SNR / DOP。見出し（`.sub-head`）と本体（`.an-cell`）を別セルにして、見出しの行数が違ってても左右の図の上端が揃うようにしている。行数の多い DRMS 集計テキストは `.an-wide` で2列ぶち抜き。列幅に追従させるため `grid-template-columns` は `minmax(0, 1fr)`（canvas の内在サイズに列幅を引っ張られないように）。
- ソースはラジオで切替。読込データが無い間は「読込データ」を `disabled` にする。
- 保存サンプル → 表示用エポックへの変換（`sampleToEpoch`）で `recvAt` を補い、`canvas-view.js` のキャリーフォワード判定（`holdDecision`）がライブと同じ経路で働くようにしている。
- 系統別内訳は、ライブでは GSA 由来の `usedBySys` / `inViewBySys` を使い、無い場合（読込データ）は衛星リストから数える。
- 比較系列は `ScatterPlotView.update(stats, deviceStats)` の第2引数で渡す。基底クラスのリサイズ再描画は `update(this._last)` と1引数で呼ぶため、第2引数の既定値を **保持中の系列**（`this._lastDevice`）にしてある（比較を消すときは明示的に `null` を渡す）。
- `update()` は表示範囲の外に出た点数 { `gnss`, `device` } を返す。呼び出し側（記録タブ・解析タブ）が「表示範囲の外側に N 点」を凡例の下に出す（`algospec-202607.md` 6-4）。
- 表示半径はヒステリシス付きでビューが保持する。**別のデータへ切り替えるときは `resetScale()` を呼ぶ**（記録開始時、解析タブのソース切替時・`setLoaded` 時）。呼ばないと前の記録のスケールを引きずる。
- 統計円のラベルは**種別のみ**（`CEP50` / `DRMS`）で、系統は色で表す。象限を分けて描くのは、比較系列を重ねるとスケールが大きい方に引かれ、受信機側の円が小さくなってラベルが中心で潰れるため。中心からの距離にも下限（10 px）を設けている。
- 系統は色だけでなく**形**でも分ける（塗り＝GNSS受信機 / 中抜き＝Android内蔵）。屋外の直射日光下と色覚特性の両方で、色だけの区別は落ちるため。凡例の記号も同じ規則。

6. モジュール構成

受信 / 解析・記録 / 画面の3層で分ける。分割の単位は「1ファイル＝1関心事」で、ファイル数そのものは目安にしない（202607 の 2 回のリファクタリングを経て 20 ファイル。8～9. 参照）。

層	モジュール	責務
エントリ	<code>app.js</code>	受信パイプラインと各タブの結線のみ + 起動時の空下書き掃除
共通	<code>constants.js</code>	設定の既定値・2 系統の表示名と色（ <code>SERIES</code> ）・容量のしきい値（ <code>STORAGE_LIMITS</code> ）
共通	<code>view-utils.js</code>	<code>\$</code> （DOM ショートハンド）・タブ切替・表示フォーマッタ・停止サマリ・測位結果テキスト
受信	<code>transport.js</code>	BLE(NUS) 接続・自動再接続 + 開発用モック（同じ <code>onFrame</code> I/F）

層	モジュール	責務
受信	nmea.js	行復元 (<code>LineBuffer</code>) → センテンス解析 (<code>parseSentence</code>) → エポック確定 (<code>EpochAssembler</code>)
受信	stream-stats.js	受信品質統計 (\$PPICO 突合を含む)
受信	device-gnss.js	Android内蔵GNSS (Geolocation API) の並行取得。記録中のみ <code>watchPosition</code>
解析・記録	accuracy.js	水平精度推定・DRMS/CEP 集計・収束判定
解析・記録	charts.js	Canvas 描画 (共通土台 + スカイプロット / SNR / 散布図)
解析・記録	recorder.js	record / stop の制御・記録中の追記・途絶停止
解析・記録	beep.js	自動停止の通知音 (AudioContext。記録ボタンで unlock)
解析・記録	survey.js	調査日→地点のID体系・測定区間・対応検証 (純粋関数のみ)
解析・記録	storage.js	IndexedDB ラッパ (下書き・チャンク追記・結合読み出し)
解析・記録	file-io.js	CSV / GPX / NMEA / JSON 出力・調査日バンドルと対応表・JSON 取込
画面	connect-ui.js	接続タブ (受信経路のライフサイクル・受信品質パネル・モック切替)
画面	record-ui.js	記録タブの収集フロー (record/stop/save/破棄・進捗バー・凡例・現在の測位値・Wake Lock)
画面	session-list-ui.js	記録一覧 (調査日ツリー・地点の読込/編集/削除・エクスポート・取込・容量警告)
画面	photo-ui.js	地点の写真 (縮小・JPEG 圧縮 + 追加/削除/サムネイルのパネル)
画面	analysis-ui.js	解析タブ (SkyPlot / SNR / DOP / DRMS、ライブ↔読込切替)
画面	map.js	地図タブ (<code>MapView</code> + タブ配線) + オフラインタイル事前DL (<code>TileCache</code> + 設定タブ内のUI)
画面	settings-ui.js	設定タブ (各種設定 + アプリ更新確認)

- `app.js` は結線のみ。DOM 操作はタブごとの `*-ui.js` が持つ。
- 読込データ (`loaded`) は `app.js` が保持し、`setLoaded()` で解析タブ・地図タブへ配る。
- 記録タブは 3 モジュールに分かれる。`record-ui.js` が入口で、そこから `session-list-ui.js` (一覧) と `photo-ui.js` (写真) を生成して束ねる。未確定の記録 (`pending`) は `record-ui.js` だけが持ち、一覧側からは `getPendingId()` / `onPendingGone()` の 2 本だけで触る。
- `device-gnss.js` は「受信経路」であってライフサイクルを持たないため、開始/停止は `Recorder` が握る (記録区間=取得区間を自明にする)。`app.js` は `onSample` → `recorder.addDeviceSample` を

結線するだけ。状態（許可待ち・拒否など）は `deviceGnss.status` を記録タブの凡例が読みに行く（通知は持たない）。

- `nmea.js` は他モジュールへ依存しない（受信経路の最下層）。
- 画面に出る系統名は `constants.js` の **SERIES** だけから引く。受信機の機種を替えたときに追従漏れが起きないようにするため、各モジュールにラベルを直書きしない（テスト側も **SERIES** を参照する）。
- 廃止：旧「ライブ」タブ（記録タブの「現在の測位値」へ統合）／旧「情報」タブ（設定タブへ統合）／スナップショット記録／記録の一時停止（打ち切りはデータ途絶で判定。3. 参照）。

7. PWA・オフラインタイル

- 版数は `sw.js` の `APP_VERSION` ('yyyy-mm-dd.n' = 更新日 + 同日内の連番。日付が変われば 1 に戻す)。シェルキャッシュ名は `gnss-scope-shell-${APP_VERSION}`（接頭辞は `activate` の掃除で他キャッシュと区別するためのもの）。`settings-ui.js` は `APP_VERSION` を正規表現で読み取り、接頭辞なしの版数で表示・比較する（旧SWがキャッシュ名を返す場合は接頭辞を外す）。`APP_SHELL` には全モジュールを列挙する。
- 地理院タイルは `cache-first` でキャッシュ `gsi-tiles` に格納する。Service Worker（実行時の取得）と事前ダウンロード（`map.js` の `TileCache`）が同じキャッシュ名を共有する点が必要。
- 未キャッシュのタイルをオフラインで要求した場合は 504 を返し、Leaflet 側でグレー表示にする。

タイルマニフェスト (`data/tile_manifest.json`)

事前ダウンロードの対象を座標リストで持つ（Format A）。マニフェストは `data/` を唯一の置き場とし、ルート直下に複製を置かない。

```
{
  "version": "2026-06-01",
  "layers": {
    "<任意のキー>": { "z": 17, "tiles": [[x, y], [x, y], ...] }
  }
}
```

- `z` はズームレベル、`tiles` は当該ズームのタイル座標 `[x, y]` の配列。
- URL は選択中の地図種別のテンプレート（`GS1_LAYERS[mapType].url`）の `{z}/{x}/{y}` を置換して生成する。したがって地図種別を変えると、その種別のタイルは再取得が必要。
- `version` は取得時に `settings.tileCacheMeta` へ記録し、オフラインでも取得済み内容を表示できるようにする。

8. リファクタリング 第1回（202607・機能変更なし）

ファイル数が多く（27）、似た処理が分散していたため、機能を変えずに統合・整理した。

8-1. ファイルの統合（27 → 17）

統合後	統合したファイル	まとめた理由
<code>nmea.js</code>	<code>nmea-parser</code> + <code>line-buffer</code>	どちらも「生バイト列 → 1センテンス」の下位層。チェックサム計算を共用できる

統合後	統合したファイル	まとめた理由
transport.js	ble-client + mock-feeder	同じ onFrame I/F で差し替える受信経路。実機／モックは対の関係
epoch.js	epoch-assembler (改名)	名前をファイルの役割に合わせた
charts.js	canvas-view + sky-plot + snr-chart + scatter-plot	すべて Canvas 描画。共通土台と3ビューを同居させ、共通ヘルパを内部化
file-io.js	exporter + importer	記録のファイル入出力。JSON 形式の定義を1か所に持てる
view-utils.js	format + tab-ui + \$ ヘルパ	画面共通のユーティリティ。\$ を各モジュールへ引数で配る配線が不要になる
map.js	map + map-ui	地図本体とタブ配線は常に一緒に変更する
tile-cache.js	tile-cache + tile-ui	タイル取得と、その進捗UIは一体
settings-ui.js	settings-ui + info-ui	アプリ更新確認は設定タブの中にある機能
record-ui.js	record-ui + wake-lock	「記録中の画面維持」は記録機能の一部
connect-ui.js	(app.js から分離・新規)	接続制御と受信品質パネルを app.js から切り出し

app.js は結線の上に絞って 330 行 → 162 行。各 UI モジュールは自分が使う描画ビューを 自前で生成するようにして、app.js からの受け渡しを減らした。

8-2. 重複処理の共通関数化

対象	変更
チェックサム XOR 計算	パーサの検証とモックの文生成で重複 → xorChecksum() に集約
HTML / XML エスケープ	escapeHtml と escXml の2実装 → escapeMarkup() に統合
キャリーフォワード判定	スカイプロットと SNR で同じ 12 行 → CanvasView._holdOrClear()
十字・リング・方位ラベル描画	スカイプロットと散布図で別実装 → drawCross / strokeCircle / fillCircle / drawCompass
タイル URL 列挙	件数カウントとダウンロードで二重ループ → TileCache.tileUrls()
設定行の保存処理	6 個のほぼ同型リスナ → 定義テーブル + 2 ループ
「使用/視野内」表示	記録タブと解析タブで重複 → satsText()
追従トグルの通知	3 か所で個別に UI 反映 → MapView.setFollow() に集約
一覧の行アクション	if 連結 → runAction() + エクスポートの対応表

8-3. 未使用コードの削除

- エポックが持っていて誰も読まない値：`geoidSep / status` (RMC) / `altStd` (GST) / `sentenceTypes / invalidCount` (不正文の件数は `stream-stats.js` が数えている)。パーサ側の対応フィールドも併せて削除。
- 使われていない設定可能性：`NmeaBle` の `deviceName / maxBackoffMs` (モジュール定数へ)、`MockFeeder` の `hz`、`TileCache` の `zoomFilter` と `countTiles()`。
- 読めないフィールド・引数：`NmeaBle.status` と `_setStatus()`、`WakeLockManager` の `isSupported()` と `onChange` の第1引数、SNR チャートの到達不能ガード。
- 外へ出す必要のない `export` の内部化：`HOLD_MS / CanvasView / stopReasonText / STOP_REASON / rxStatsText / JSON_FORMAT`。

なお `computeStaticStats` の `avgPdop / avgVdop` は画面に出ないが、**JSON エクスポートで利用者へ渡る値**なので残している。

9. リファクタリング 第2回 (202608・機能変更なし)

第1回の後に写真・調査日ツリー・容量管理が乗り、`record-ui.js` が 949 行まで肥大化していた (次に大きい `charts.js` の 2 倍)。**ファイル数ではなく1ファイルの守備範囲**が問題だったため、肥大化したものを割り、常に一緒に変更するものを寄せた。結果としてファイル数は 22 → 20。

9-1. 記録タブの分割 (949 行 → 3 ファイル)

`record-ui.js` は「収集フロー」「記録一覧」「写真」の 3 つを抱えていた。一覧の描画を直すたびに収集フローのコードを読み飛ばす必要があり、変更の単位と合っていない。

分割後	行数	責務
<code>record-ui.js</code>	約 400	record / stop / save / 破棄・進捗バー・凡例・現在の測位値・Wake Lock
<code>session-list-ui.js</code>	約 330	調査日ツリー・地点の読込/編集/削除・エクスポート・取込・容量警告・下書きの確定
<code>photo-ui.js</code>	約 160	写真の縮小 (旧 <code>photos.js</code>) + 追加/削除/サムネイルのパネル

未確定の記録 (`pending`) は `record-ui.js` だけが持ち、一覧側へは `getPendingId()` / `onPendingGone()` の 2 本だけを渡す (状態の持ち主を 1 か所に留める)。写真パネルは保存フォーム (静的な要素) と一覧の編集フォーム (行ごとに組み立てる文字列) の 2 か所に出るため、`wirePanel()` と `panelHtml()` の 2 つの入口を持たせて実装は共有した。

9-2. ファイルの統合 (22 → 20)

統合後	統合したファイル	まとめた理由
<code>nmea.js</code>	<code>nmea + epoch</code>	「チャンク → 行 → センテンス → エポック」は一本の流れで、常に一緒に使う
<code>map.js</code>	<code>map + tile-cache</code>	どちらも <code>GSI_LAYERS</code> と地図種別を見る。地図ドメインの外へ出す必要がない

統合後	統合したファイル	まとめた理由
photo-ui.js	photos + 記録タブの写真配線	縮小と取り込み UI は対。58 行の変換だけを単独ファイルにする理由がない

9-3. 重複処理の共通関数化

対象	変更
測位結果テキストの組み立て	停止直後・保存済み・解析タブの 3 か所で同じ並び → <code>formatResult()</code>
「表示範囲の外側に n 点」	記録タブと解析タブで同一実装 → <code>renderOutside()</code>
ローカル時刻 <code>yyyy-mm-dd HH:MM:SS</code>	<code>file-io</code> の <code>localStamp</code> と <code>view-utils</code> の <code>localTime</code> → <code>view-utils</code> へ集約
中央値	<code>accuracy</code> の集計と <code>survey</code> の時計オフセットで別実装 → <code>median()</code> を共有
実測の長さ [秒]	<code>recorder.spanSecOf</code> は <code>survey.timeWindow</code> の焼き直し → <code>spanSec()</code>
IndexedDB の定型 3 手	<code>_store</code> / <code>_get</code> / <code>_getAll</code> / <code>_byIndex</code> / <code>_put</code> / <code>_delete</code> に畳む (10 か所)
調査日レコードの生成・更新	<code>ensureSurvey</code> と <code>createDraft</code> で重複 → <code>_surveyRecord()</code> / <code>_patchSurvey()</code>
写真の増減と <code>summary</code> 更新	<code>addPhoto</code> / <code>deletePhoto</code> で同型 → <code>_changePhotos()</code>
CSV のコメント塊	「地点 / 集計値 / 比較」で 3 実装 → <code>commentBlock()</code>
中抜きの際の円の描画	スカイプロットと散布図で <code>arc</code> + <code>stroke</code> 直書き → <code>strokeCircle()</code> に線幅引数
設定行の配線	種類ごとに 3 ループ → 定義テーブル + 1 ループ
写真の長辺の選択肢	<code>index.html</code> に直書き + <code>PHOTO_EDGE_OPTIONS</code> の二重管理 → 定数から生成

9-4. 未使用コードの削除

- `survey.js` の `PAIRING_LABELS` (`pairingOf` の判定結果だけを使っている)。
- `DeviceGnss` の `isRunning` と `onStatus` 通知。状態は `deviceGnss.status` を記録タブの凡例が読むだけで足りる (受け手の `recordUI.onDeviceStatus` は空実装だった)。
- `Recorder.onUpdate` の `rawLines` (受け手が分解していない)。
- `initTabUI` が返す `show` (外から呼んでいない)。
- `drawStatCircle` の `ctx.lineWidth = 1.5` (直後の `strokeCircle` が 1 で上書きしていた死んだ行)。

10. テスト

`node tools/check-syntax.mjs` に以下を追加した。

- 使用衛星の平均 C/N0（未使用衛星を除外すること）
- `formatStats` が未確定の記録でも生成できること
- 記録フロー：開始時に下書き（`status:'draft' / pointNo:null`）を作ること／5 エポックごとにチャンクを追記し進捗を下書きへ残すこと／`stop` で `summary` に集計値が入っても下書きのままであること／チャンクを結合すると全エポックが揃うこと／確定で地点番号を採番すること
- 下書き：地点番号を消費しないこと／削除でチャンクも消えること／0 点は自動削除されること／追記が連続 3 回失敗したら打ち切ること
- データ不足の判定（30 秒 / 10 点。0 点・30 秒未満は別扱い）
- 端末内のデータ量：停止時に概算バイト数を残すこと／`formatBytes` の MB・KB 表示／写真の増減で `photoCount / photoBytes` が動くこと／地点を消せば写真も消えること
- 書き出し記録：`markExported` で書き出し済みになること／書き出し済みは未書き出し量に数えないこと／写真を足すと未書き出しへ戻ること（`updatedAt` は必ず前の値より進める）
- Android内蔵GNSSの比較：`offsetBetween` の距離と方位／`computeDeviceStats` の座標重複数・平均 accuracy・中心ズレ／`formatCompare` の主要行と過小評価の注意／`withDevice` の ON・OFF で取得の開始停止と保存形（OFF なら従来どおりのキー構成）が変わること
- 収束の進捗：フェーズ1/2 の切替（`windowReady`）・`stableSec`・許容値の返却
- 停止サマリ：5 つの停止理由の文言・「未収束」と「収束判定なし」の書き分け・点数0・`autoStop` を持たない旧データ・`formatStats` が停止理由を含まないこと

第2回のリファクタリングで共通化した関数にも足した。

- `formatResult`：集計+比較+測定区間の並び／比較データが無ければ比較ブロックを出さないこと／`window` を渡さない（ライブ表示）と測定区間を出さないこと
- `spanSec`：`recvAt` を優先して範囲を測ること／0~1点は0になること
- `localStamp`：`yyyy-mm-dd HH:MM:SS` の整形と、`null` が空セルになること

表示文言のテストは `SERIES` を参照して書く（ラベルを直書きすると、機種を替えて `constants.js` を直したときにテストだけ古い名前で落ちる）。

DOM を伴う経路（タブ切替・記録一覧・写真パネル・設定行）は Node 側のテストでは追えないため、`headless Chrome` で `record` → `stop` → `save`、下書きの確定、破棄、地点の編集・削除、写真上限 0 での UI 消失、設定値の丸めまでを一巡させて確認する。